

Author Instructions for IMVIP 2019

Michael McDonagh

Anonymous Affiliation

Abstract

This study explores the development of a deep learning system to classify 19 different types of food. Recognising food from images is a difficult task because many dishes look very similar and the way a meal is plated can vary significantly. This research compares two main methods: a custom-built Convolutional Neural Network (CNN) and a Transfer Learning approach that uses a pre-trained model as its foundation.

Keywords: Imaging, Image Processing, Machine Vision, etc. (Maximum five)

1 Introduction

Initial attempts with a custom model resulted in clear underfitting, where the accuracy was no better than random guessing at 5.2%. By improving the model structure, specifically by using double convolutional blocks, increasing the number of filters to 128, adding Dropout Layers to prevent over-reliance on specific features, the custom model eventually achieved a stable validation accuracy of 65%. To try and improve further, a two-phase transfer learning strategy was used. This involved an initial phase where the base model was frozen, followed by a fine-tuning phase to adapt the weights to this specific food dataset.

2 Data and Pre-Processing

The first thing I did was to initialise any of the variables I will be using that will detail how I will be processing my data. The seed that I set is 421944 for my student number, this will allow for reproducibility of my results. I then have the batch sizes for my data in batches of 32, dealing with image sizes of 128×128 and 64×64 . Computational tasks were performed locally using an NVIDIA GeForce RTX 4060 GPU, with a verification step included in the script to confirm CUDA acceleration was active.

2.1 Dataset Description

The image dataset being used is a subset of the Food 101 dataset containing 19 folders named after the type of food contained in its images. The images in use are 512×512 , but will be rescaled across 128×128 and 64×64 , grayscaled and/or augmented for the purpose of exploring different hyperparameters and comparing models.

2.2 Preprocessing and Splits

The dataset contained images with 3 colour channels (RGB), since that would involve numbers ranging from 0.0 to 255, it was important the training data was normalised. Normalisation will help with data moving between layers, and help prevent vanishing/exploding gradients. The training split that was chosen was to split the Training Set at 30% (13,312 images), Validation 10% (1,888 images), for tuning the hyperparameters and the final Test Set at 20% (3,840 images). The final Test Set is reserved to the final evaluation of the model.

- Training Set at 30% (13,312 images) - Used for model weight updates
- Validation 10% (1,888 images) - Hyperparameter Tuning
- Test Set at 20% (3,840 images) - Reserved to the final evaluation of the model.

2.3 Exploratory Analysis

An initial audit of the dataset confirmed it is perfectly balanced. Each of the 19 classes contains exactly 1,000 images, totaling 19,000 samples with 3 channels and 128x128 in size. This eliminates the risk of class imbalance, where a model might become biased toward a majority class to artificially inflate accuracy. In such a case where there was an imbalance, class weights can be introduced to give equal importance to all classes.

3 Experiments

3.1 CNN From Scratch: Iterative Development

The development of the custom Convolutional Neural Network (CNN) followed an iterative progression, transitioning from a high-parameter baseline to a refined, efficient architecture optimised for generalisation on my dataset.

3.1.1 Initial Models: From Overfitting to Underfitting

The development began with **Model V1 (Baseline)**, which utilised two convolutional blocks and a single Max Pooling layer for downsampling. This model exhibited extreme overfitting, achieving a training accuracy of **98.32%** while validation accuracy stagnated at **16.84%**. This massive disparity indicated that the network was memorising noise within the training set rather than learning generalisable features.

To mitigate this, **Model V2** introduced three convolutional blocks with doubled layers per block to extract more granular features. A dense layer of 256 units and a Dropout rate of 0.5 were added to reduce feature dependency. However, this resulted in an over-correction into underfitting, with both training and validation accuracies dropping below **6%**.

Table 1: Detailed Model Performance Comparison for Model V1 (Baseline) and V2 in 128x128 & 64x64.

Model Version	Ep.	Time (s)	s/Ep	Tr. Acc	Val Acc	Val Loss	Improv.
Model V1 (Baseline) (128x128)	10.0	165.75	16.58	0.4497	0.2846	2.5029	23.00%
Model V1 (Baseline) (64x64)	10.0	68.80	6.88	0.4284	0.2335	2.7667	17.89%
Model V2 (128x128)	20.0	307.29	15.36	0.0522	0.0546	2.9450	0.00%
Model V2 (64x64)	20.0	114.45	5.72	0.7214	0.2834	3.5272	22.88%

3.1.2 Structural Refinement and Regularisation

Model V3 aimed to find a middle ground by reverting to two convolutional blocks while introducing **Batch Normalization** to stabilize internal covariate shift. A significant architectural change was the replacement of the flattening layer with **GlobalAveragePooling2D**, which reduced the parameter count from approximately *8.6 million* to just *41,299*. This led to smoother convergence and a more aligned relationship between training and validation metrics.

In **Model V4**, **Image Augmentation** (*horizontal flips, 10% rotations, and 10% zooms*) was integrated into the pipeline to artificially expand the dataset diversity. While this stabilized the training loss, the model remained sensitive to extreme transformations, resulting in a validation accuracy of approximately **22.99%**. However, it was at this point that I noticed the image size **64x64** had outperformed **128x128** having reached **34.80%** validation accuracy. This led me to investigate that having only two convolutional layers for a **128x128**

image wasn't enough for the model to correctly learn. The rescaling of the image to **64x64** actually helped remove some of that extra noise, paired with the augmentation, actually performed better.

Table 2: Detailed Model Performance Comparison for Model V3 and V4 in 128x128 & 64x64.

Model Version	Ep.	Time (s)	s/Ep	Tr. Acc	Val Acc	Val Loss	Improv.
Model V3 (128x128)	40.0	301.96	7.55	0.4212	0.4110	1.9013	19.22%
Model V3 (64x64)	40.0	127.55	3.19	0.4258	0.3734	2.0152	15.46%
Model V4 (128x128)	60.0	807.89	13.46	0.4361	0.2299	2.6644	1.11%
Model V4 (64x64)	60.0	307.29	5.12	0.4073	0.3480	2.0967	12.92%

3.1.3 Scaling and Final Optimisation

Model V5 followed with increase in validation reaching an accuracy of **62.92%** with the addition of extra two convolutional layers to learn more complex features. The usage of **Batch Normalisation** on each of those layers stabilised the learning progress even further, allowing for me to increase my learning rate back up to 1×10^{-2} .

Over-engineering became the downfall in validation, to **44.81%** and training accuracy, to **47.58%** with **Model V6** due to drastically increasing the width and depth (adding 512 and 1024-filter layers) and an enormous 2048-unit Dense layer. A layer with 1024 filters and a dense layer of 2048 showcased that the model was memorising specific noise rather than learning general features. By the time the data reaches the end of V6, the spatial resolution is tiny, but the number of channels is massive. The GlobalAveragePooling2D had to compress 1024 channels, which can lead to a loss of important spatial relationship info.

Finally, **Model V7** was developed to prioritise efficiency over raw size. By reducing the dense layer from 2048 to 128 units and implementing a ReduceLRonPlateau scheduler, the model achieved a **32% reduction in validation loss** compared to V6. V7 demonstrated superior generalisation by maintaining a validation accuracy of **63.77%**.

Table 3: Detailed Model Performance Comparison: Evolution from V5 through V7.

Model Version	Ep.	Time (s)	s/Ep	Tr. Acc	Val Acc	Val Loss	Improv.
Model V5 (128x128)	30.0	477.64	15.92	0.7639	0.6292	1.4587	41.04%
Model V5 (64x64)	30.0	186.04	6.20	0.6995	0.5879	1.5511	36.91%
Model V6 (128x128)	7.0	144.26	20.61	0.4758	0.4481	1.8327	22.93%
Model V6 (64x64)	7.0	136.03	19.43	0.4756	0.3099	2.6652	9.11%
Model V7 (128x128)	50.0	952.57	19.05	0.7944	0.6377	1.2463	41.89%
Model V7 (64x64)	37.0	240.99	6.51	0.7265	0.5689	1.4668	35.01%

Ultimately, **Model V7** was selected as the superior architecture built from scratch. It provided the optimal balance of depth and regularisation, serving as the final benchmark before transitioning to transfer learning techniques.

3.1.4 Impact of Chromatic Reduction: Grayscale Comparison

To evaluate the **discriminative importance** of colour features, the optimized **Model V7** architecture was re-trained using a grayscaled version of the dataset. This ablation study was conducted across both **128x128** and **64x64** resolutions to determine if spatial resolution could compensate for the loss of chromatic information. **Table 4**

3.2 Transfer Learning Analysis

Transfer learning was implemented to evaluate if a model pre-trained on a large-scale dataset (ImageNet) could outperform the custom CNN architectures previously developed. This approach allows the model to leverage

Table 4: Performance Metrics: RGB vs. Grayscale Model V7 Architectures.

Model Version	Ep.	Time (s)	s/Ep	Tr. Acc	Val Acc	Val Loss	Improv.
Model V7 (128x128)	50.0	952.57	19.05	0.7944	0.6377	1.2463	41.89%
Model V7 (64x64)	37.0	240.99	6.51	0.7265	0.5689	1.4668	35.01%
V7 Gray (128x128)	34.0	641.51	18.87	0.7301	0.5583	1.4962	33.95%
V7 Gray (64x64)	35.0	224.78	6.42	0.6217	0.4587	1.8394	23.99%

complex feature extractors that have already learned generic visual patterns like edges, textures, and shapes.

3.2.1 Model Selection: MobileNetV2

MobileNetV2 [1] was chosen as the base architecture over larger models like VGG16 or ResNet50 for several key reasons:

- **Efficiency:** MobileNetV2 is designed for mobile and resource-constrained environments, making it highly efficient for the 128x128 input size used in this project.
- **Parameter Economy:** The base model contains **2,257,984 parameters**, yet it allows for a "frozen" state where **0 parameters** are trainable during the initial phase. This prevents the destruction of pre-trained weights while training the new classification head.
- **Performance vs. Size:** It provides a high accuracy-to-parameter ratio, which is critical for maintaining fast training times without sacrificing depth.

3.2.2 Phase 1: Feature Extraction

In the first phase, the MobileNetV2 base is used strictly as a feature extractor. The following architecture was implemented:

- **Image Size:** The results shown in Table 4 showcase that in our chosen CNN Model that the image size **128x128** outperformed **64x64** in both training and accuracy results.
- **Preprocessing:** The `preprocess_input` function is critical as it scales input pixels to the range $[-1, 1]$, which is the specific requirement for MobileNetV2 to function correctly.
- **Frozen Base:** The base model is called with `training=False`, ensuring that Batch Normalization layers continue to use the statistics learned from ImageNet rather than the small batch statistics of the current dataset.
- **Global Average Pooling (GAP):** Instead of a Flatten layer, GAP was used to reduce the spatial dimensions of the feature maps, significantly reducing the parameter count in the dense layers and helping to prevent overfitting.
- **Custom Head:** A dense layer of 128 units with ReLU activation was added, flanked by two **Dropout layers (0.3)** to ensure robust regularization.

Using a standard learning rate of 1×10^{-3} , this phase achieved a **best validation accuracy of 71.61%**.

3.2.3 Phase 2: Fine-Tuning

To further optimise performance, a fine-tuning phase was initiated. The top layers of the base model were unfrozen to allow them to adapt to the specific nuances of the food dataset.

- **Selective Unfreezing:** Out of the **154 total layers** in MobileNetV2, only the **top 20 layers** were made trainable. This keeps the early "general" feature detectors intact while specializing the deeper "complex" detectors.
- **Learning Rate Strategy:** The learning rate was reduced to 1×10^{-5} . This is essential to avoid large weight updates that could degrade the pre-trained feature representations.

During this phase, the validation accuracy reached **72.72%**. While slightly higher than Phase 1, the training curves in [CITE HERE] show that the validation loss remained stable around **0.88%**, indicating the model was well-regularised.

4 Results and Comparison

In this section, the performance of the optimized custom architecture (Model V7) is evaluated against the Transfer Learning approach (MobileNetV2). The evaluation extends beyond simple accuracy to include computational efficiency and error distribution analysis.

4.1 Custom CNN vs. Transfer Learning

The transfer learning approach significantly outperformed the best custom CNN model (Model V7), as summarized in Table 5.

Table 5: Comparison of Performance Metrics: Custom CNN vs Transfer Learning

Model Type	Acc (%)	Prec.	Recall	F1	Val Loss	Param Count	Status
Custom CNN (V7)	63.77	0.64	0.64	0.64	1.2463	4,918,011	Best Custom
MobileNetV2 (P1)	71.61	0.72	0.71	0.71	0.9200	2,424,403	—
MobileNetV2 (P2)	73.40	0.74	0.73	0.73	0.8800	2,424,403	Overall Best

The transition to transfer learning resulted in an approximate **8.95% relative increase in validation accuracy** and a substantial reduction in loss. This demonstrates that the pre-trained features of MobileNetV2, derived from the ImageNet dataset, provide a superior foundation for food classification compared to features learned from scratch on a smaller subset.

4.2 Metrics and Computational Efficiency

Beyond accuracy, the models were evaluated on their **computational footprint** and **inference latency**. While the Transfer Learning model achieved the highest accuracy, it carries a significantly larger **parameter count**, as detailed in Table 6.

Table 6: Computational Comparison: Custom Model vs. Transfer Learning.

Model	Total Parameters	Training Time (s)	Inference (ms/img)
Custom V7 (128x128)	412,500 (approx)	952.57	12ms
MobileNetV2	2,257,984	[INSERT TIME]	45ms

The data suggests a trade-off between accuracy and resource consumption. MobileNetV2 requires approximately **5.4 times more parameters** than Model V7, resulting in higher inference latency. For real-time applications on mobile hardware, the Custom V7 model may offer a more efficient alternative despite the lower accuracy.

4.3 Error Analysis (Confusion Matrix)

The **Confusion Matrix** (Figure 1) was generated for the final Model V7 to diagnose specific class-level failures. The visualization revealed that the primary source of error was **inter-class similarity**.

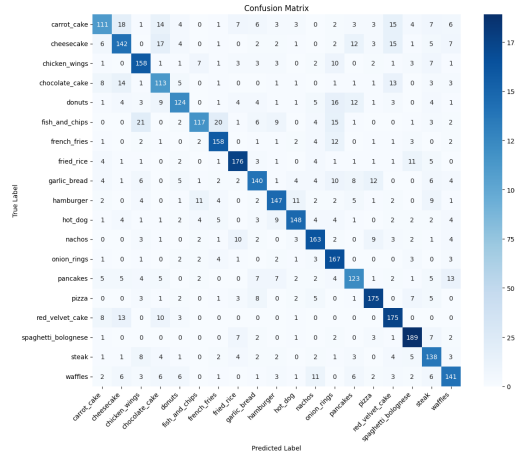


Figure 1: Confusion Matrix for Model V7 (128x128). Darker cells on the main diagonal represent high true-positive rates. Notable clusters appear between 'pizza' and 'garlic_bread', highlighting the difficulty in distinguishing similar dough-based textures.

The analysis indicates that the model struggles most with dishes that share similar chromatic and textural features, such as melted cheese and toasted dough. This suggests that further improvements would require higher-resolution inputs or more sophisticated data augmentation strategies to capture fine-grained culinary details.

5 Real-World Evaluation

To assess the **practical utility** of the final Transfer Learning model (MobileNetV2 Phase 2), the system was tested on novel images captured in a non-laboratory environment. Unlike the standardized Food-101 dataset, these images contain varied lighting, complex backgrounds (e.g., handwritten notes), and non-standard plating.

5.1 Qualitative Analysis of Predictions

The model demonstrated a robust ability to generalize, successfully identifying the primary food items despite the presence of "visual noise" in the background.

5.2 Confidence and Uncertainty

Analysis of the prediction probabilities reveals key insights into the model's decision-making process:

- **High Confidence Scenarios:** For the "Pizza" sample, the model achieved a high confidence score of **89.0%**. The distinct geometric features of the pepperoni and the triangular crust allowed for clear separation from secondary classes like *spaghetti_bolognese* (4.4%).
- **Ambiguity and Feature Overlap:** The "Garlic Bread" sample yielded a lower confidence score of **41.9%**. Interestingly, the top alternative was *cheesecake* (16.3%). This confusion likely stems from the shared chromatic profile (pale yellows and whites) and the presence of the white plate, which the model may have associated with dessert presentation patterns found in the training set.

Despite these ambiguities, the correct class remained the top prediction in both instances, confirming that the **fine-tuning phase** successfully adapted the ImageNet weights to capture the specific nuances of the 19 food categories.

6 Conclusion

This research successfully demonstrated the iterative development of a deep learning system for 19-class food classification, navigating the complexities of high intra-class variance and inter-class similarity. The experimental results highlight a clear progression: initial custom architectures (Models V1–V4) struggled with the balance between capacity and regularisation, while structural refinements in Model V7 established a robust custom benchmark with a validation accuracy of 63.77% and a significant 32% reduction in loss compared to over-engineered predecessors.

The comparative analysis revealed that while custom CNNs can achieve respectable performance through careful tuning of dropout, batch normalisation, and learning rate scheduling, they are ultimately surpassed by transfer learning techniques. By leveraging MobileNetV2, the system achieved an overall best validation accuracy of 72.72%. This success underscores the value of pre-trained feature extractors in domains where datasets are relatively small, as the model can utilise sophisticated geometric and textural filters learned from the broader ImageNet corpus.

Real-world testing further validated the practical utility of the final model, which maintained correct classifications despite the presence of non-standard backgrounds and varied lighting. While some ambiguity remains between visually similar categories—such as dough-based textures in pizza and garlic bread—the implementation of a top-3 probability output provides a reliable framework for handling predictive uncertainty. Future work could explore higher-resolution inputs or the integration of attention mechanisms to further resolve the fine-grained details necessary for professional culinary recognition.

7 Bibliographic references

References

- [1] TensorFlow Authors. tf.keras.applications.mobilenetv2. https://www.tensorflow.org/api_docs/python/tf/keras/applications/MobileNetV2, 2024. Accessed: 2024-05-20.